WEB DEVELOPMENT · 3RD SEMESTER

Week 04 · Block 3d

Layout & Responsive Design

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Pin single elements in place with **positioning** (`relative`, `absolute`, `sticky`)
- Arrange boxes in a row with **Flexbox**
- Let a flex row **wrap** into a card layout with `flex-wrap` and `flex`
- Make a page **responsive** with the viewport meta and `@media`
- Work **mobile-first**

Every page so far has been one column, top to bottom.

Real layouts put boxes side by side.

Positioning pins single boxes · Flexbox lines them up · media queries make it fit any screen.

PART 1

Normal Flow

How boxes flow by default

- **Block** elements (`div`, `p`, `section`, `h1`) stack **vertically**, each on its own line
- **Inline** elements (`a`, `span`, `strong`) sit **in a line**, left to right
- That's **normal flow** — perfect for a document, not for a layout

Block or inline is only the **default** and can be change via the CSS property `display`.

PART 2

Positioning — pin things in place

The position property

`position`	Stays in the flow?	`top` / `left` / … measured from
`static` *(default)*	yes	— (ignored)
`relative`	yes, keeps its space	its normal spot
`absolute`	**no**	the nearest **positioned** ancestor
`fixed`	**no**	the browser window
`sticky`	yes	scrolls along, then sticks to the window

A **positioned** element is one with any `position` other than `static`.

relative + absolute: a badge on a card

```
.card { position: relative; }   /* the anchor */
.badge {
  position: absolute;           /* out of the flow */
  top: 0.5rem;
  right: 0.5rem;
}
```

- `absolute` takes the badge out of the flow — the text around it ignores it
- It measures `top` / `right` from the nearest **positioned** ancestor: the card
- `relative` without offsets does not move the card — it only makes it the anchor
- Forget it, and the badge jumps to the corner of the **page**

Sticky headers & z-index

```css
.nav {
  position: sticky;
  top: 0;          /* stick when it reaches the top */
  z-index: 1;      /* stay above the content */
}
```

- `sticky` scrolls with the page, then **sticks** once it reaches `top: 0`
- `fixed` never scrolls — but it leaves the flow, so the content slides **under** it
- `z-index` — when positioned boxes overlap, the higher value is on top

Positioning moves **single** elements: a badge, a header, a pop-up. To lay out a page, use **Flexbox**.

PART 3

Flexbox — line things up

One dimension: a row (or a column)

```
.row {
  display: flex;
  gap: 1rem;
}
```

- `display: flex` on a container → its children line up in a **row**
- **One dimension** at a time — a row, or a column with `flex-direction: column`
- `gap` spaces the children evenly

Aligning with flex

```css
.nav {
  display: flex;
  justify-content: space-between;   /* along the row  */
  align-items: center;              /* across the row */
}
```

- **`justify-content`** — position along the **main axis** (`start`, `center`, `space-between`…)
- **`align-items`** — align on the **cross axis** (here: vertically centered)

Flexbox is the go-to for **navbars, button rows, toolbars**, and centering a thing — anything that lives on a single line.

Wrapping into more lines

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
.card { flex: 1 1 200px; }   /* grow, shrink, basis */
```

- `flex-wrap: wrap` — when a row is full, the next item starts a new row
- `flex: 1 1 200px` — each card starts at **200px** (the basis), then **grows** to fill the free space
- Rows of cards that look like a grid — built from one-dimensional rows

PART 4

Responsive Design

One layout doesn’t fit every screen

Three cards in a row look fine on a laptop. On a phone they are **cramped**.

Responsive design = the layout **adapts** to the screen it's on.

Two ingredients:

1. Tell the phone to stop lying about its width — the **viewport meta**
2. Change styles by screen width — **media queries**

The viewport meta tag

```
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

- Goes in every page's `<head>`
- Without it, phones **pretend to be ~980px wide** and shrink the whole page to fit
- You've seen it in the starter files — this is what it does

A CSS pixel is not a screen pixel. Most phones are 360–430 CSS pixels wide.

Example: A Google Pixel 5 has 1080 screen pixels across, but it is **393 CSS pixels** wide. Each CSS pixel there uses about 3 screen pixels, so text stays readable.

Media queries — adapt by width

```css
/* base = mobile-first: one card per row */
.card { flex: 1 1 100%; }

/* wider screens: three cards per row */
@media (min-width: 700px) {
  .card { flex-basis: 30%; }
}
```

- `@media (min-width: 700px) { … }` applies **only above** that width
- **Mobile-first**: write the phone layout as the base, then *add* to it for bigger screens
- The width where things change is a **breakpoint**
- Why `30%` and not `33%`? The gaps need room too — and `grow` fills the rest

Responsive with no media query

```
.cards { display: flex; flex-wrap: wrap; gap: 1rem; }
.card  { flex: 1 1 200px; }
```

- You already know this one: the **wrapping row** from Part 3
- Narrow screen → fewer 200px cards fit in a row; wide screen → more
- The row **reflows on its own** — no breakpoint needed 🎉
- One catch: a last row with fewer cards **stretches** them wider than the rest

Recap

- Normal flow stacks blocks; **positioning** pins single boxes, **Flexbox** arranges them
- **Positioning**: `relative` + `absolute` (badge), `sticky` (header), `z-index`
- **Flexbox**: `display:flex`, `gap`, `justify-content`, `align-items` — rows & navbars
- **Wrapping**: `flex-wrap: wrap` + `flex: 1 1 <basis>` — a card layout from rows
- **Responsive**: the **viewport meta** + `@media` breakpoints, worked **mobile-first**

<EXERCISE> · 45 MIN

Your turn 👉

Build a **responsive card layout** — positioning for the details, Flexbox for the layout.

1. **Position** things: a sticky nav and a "New" badge on a card
2. A **flex** navbar: logo left, links right (`justify-content: space-between`)
3. A **wrapping** card row: `display: flex` + `flex-wrap: wrap` + `gap`
4. Make it **mobile-first**: 1 card per row on phones → 2 → 3 via `@media` breakpoints
5. Then replace the breakpoints with `flex: 1 1 200px` and resize the window

Starter: `exercises/week04/3d/` — open `README.md`. A nav and a set of cards are ready to lay out.